

操作系统课程设计，2025春

Unix V6++ 字符设备管理

(下 键盘 和 应用程序的标准输入输出)

同济大学计算机系

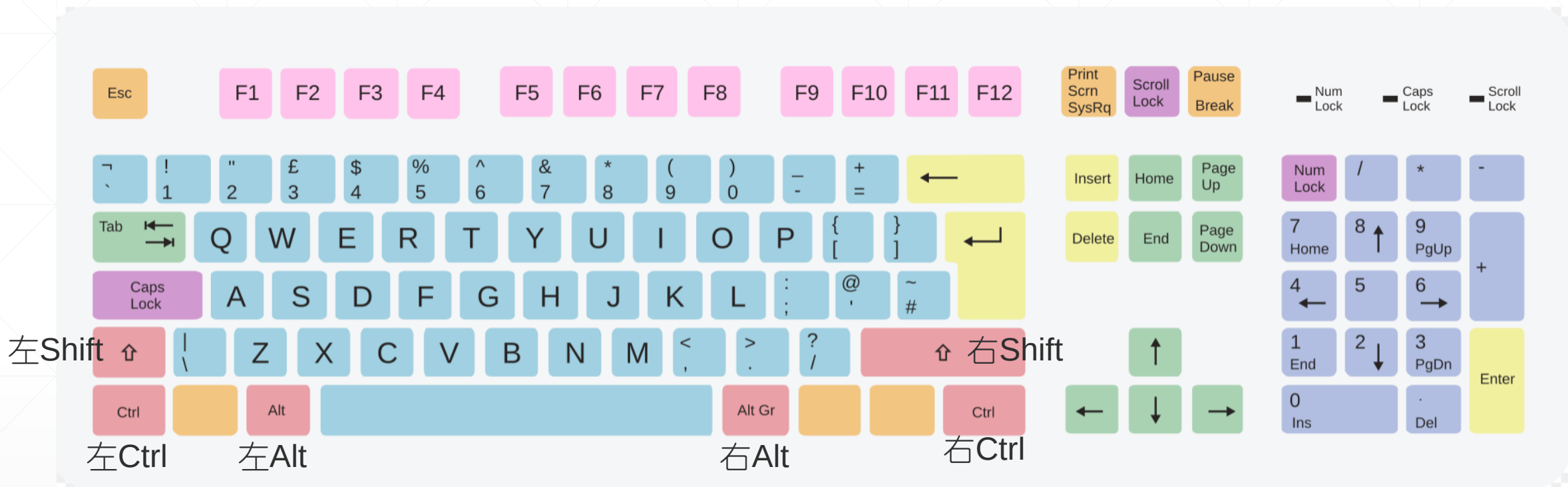


Part 2、键盘

目录

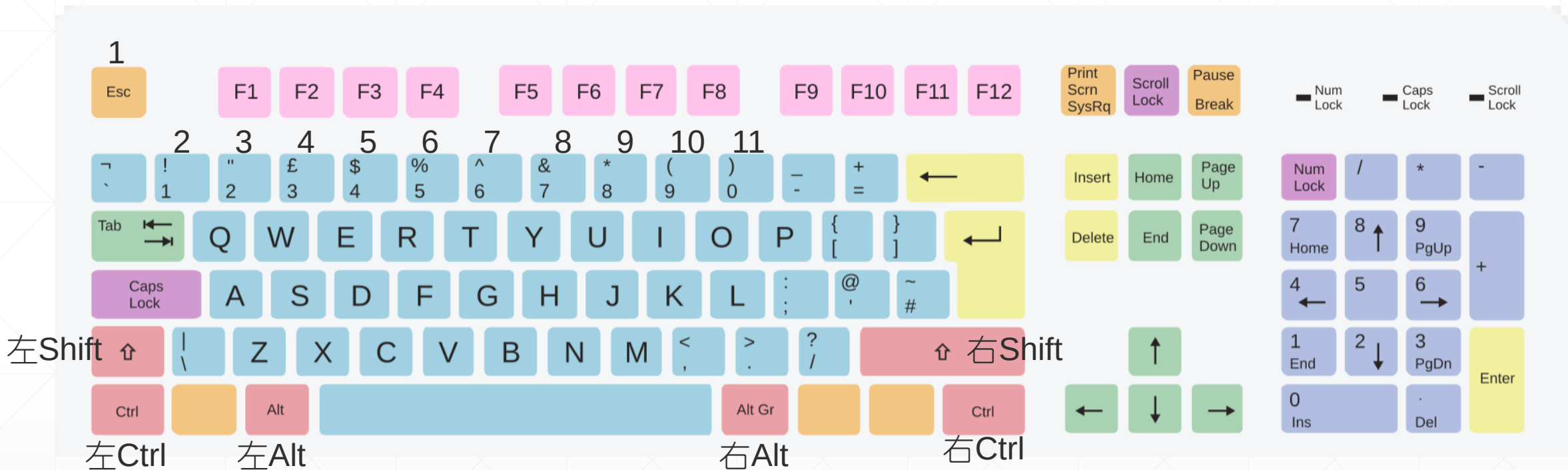
- 硬件工作原理
- 键盘中断处理程序

键盘



- | | | | |
|--|--|---|--|
|  Character keys |  Enter and editing keys |  Navigation keys |  Numeric keypad |
|  Modifier keys |  System and GUI keys |  Function keys |  Lock keys |

扫描码



每个键对应一个扫描码。譬如，数字 1 ~ 9，扫描码分别为 2 ~ 10。

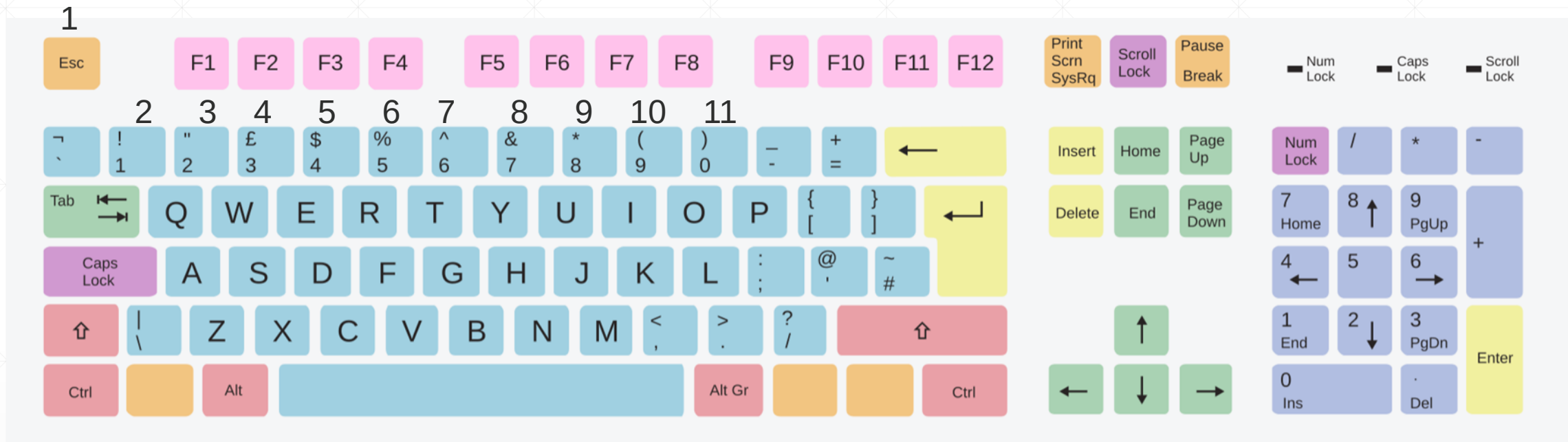
键盘的硬件接口提供的数据是扫描码（ScanCode），分为MakeCode和BreakCode，对应于一个键的按下和释放。

$$\text{BreakCode} = \text{MakeCode} | 0x80$$

举个例子，按下A键，硬件给出的扫描码是0x1e（MakeCode）；释放A键，硬件给出的扫描码是0x9e（BreakCode）。

所有按键，包括 Shift, Ctrl……均有2个扫描码

键盘中断 和 中断处理程序



按键，键盘发中断请求，给出通码 (MakeCode)。松开，键盘发第二个中断请求，给出断码。BreakCode。

键盘控制器产生键盘中断的同时，(1)送扫描码入数据寄存器 (端口号：0x60)

(2)修改状态寄存器 (端口号：0x64) 标记数据寄存器中有新数据。

键盘中断处理程序，查转换表，把扫描码转换成 控制码 和 字符码 (ASCII码)。之后，用控制码维护键盘工作模式、实施进程控制；将字符码送入原始队列。

普通字符的输入 1

1、输入字符 a 涉及2次中断请求。

第一次：键盘中断处理程序获得的扫描码是0x1e（按键A的MakeCode）。

第二次：键盘中断处理程序获得的扫描码是0x9e（按键A的BreakCode）。

2、输入字符 A（使用左Shift） 涉及4次中断请求。

第一次：键盘中断处理程序获得的扫描码是0x2a（按键Shift的MakeCode）。

第二次：键盘中断处理程序获得的扫描码是0x1e（按键A的MakeCode）。

第三次：键盘中断处理程序获得的扫描码是0x9e（按键A的BreakCode）。

第四次：键盘中断处理程序获得的扫描码是0xaa（按键Shift的BreakCode）。

普通字符的输入 2

3、输入字符“ABCD”有两种方法

(1) 按下 <CapsLock> 键后, 再输入 abcd 键, 再按 <CapsLock> 键

一个 <CapsLock> 键 make 和 break 码, 然后依次是 abcd 各自的 make 和 break 码, 最后是 <CapsLock> 的 make 和 break 码

(2) 按着 <L-shift> 或 <R-shift> 不放, 再按 abcd 键, 再放开 <L-shift> 或 <R-shift> 键

一个 <L-shift> 或 <R-shift> 键的 make 码, 然后依次是 abcd 各自的 make 和 break 码, 最后是一个 <L-shift> 或 <R-shift> 键的 break 码



按键，扫描码 和 字符码 (没有按下Shift键， make code)

```
char Keyboard::Keymap[] = {
    0,0x1b,'1','2','3','4','5','6',    /* 0x00-0x07    0, <esc>,1,2,3,4,5,6, */
    '7','8','9','0','-','=',0x8,0x9,    /* 0x08-0x0f    7,8,9,0,-,=,<backspace>,<tab>, */
    'q','w','e','r','t','y','u','i',    /* 0x10-0x17    q,w,e,r,t,y,u,i, */
    'o','p','[',']','\n',0,'a','s',    /* 0x18-0x1f    o,p,[,], <enter>,<ctrl>,a,s, */
    'd','f','g','h','j','k','l',',',    /* 0x20-0x27    d,f,g,h,j,k,l,;, */
    '\",'`,`',0,'\\','z','x','c','v',    /* 0x28-0x2f    ``,`,<lshift>,\,z,d,c,v, */
    'b','n','m',',',',',',/,0,'*',    /* 0x30-0x37    b,n,m,,,,/,<rshift>,<printscr>, */
    0,',',0,0,0,0,0,0,    /* 0x38-0x3f    <alt>,<space>,<caps>,<f1>,<f2>,<f3>,<f4>,<f5>, */
    0,0,0,0,0,0,0,'7',    /* 0x40-0x47    <f0>,<,>,<,>,<,>,<,>,<numlock>,<scrlock>,7, */
    '8','9','-','4','5','6','+','1',    /* 0x48-x04f    8,9,-,4,5,6,+,1, */
    '2','3','0','.',',',0,0,0,0    /* 0x50-0x57    2,3,0,.,,<,>,<,>,<,>,<,>, */
};
```

// 红色是按键，黑色是扫描码，金色是字符码，它们一一对应。譬如，按键1的扫描码是0x2，字符码是ASCII码，值是0x30。用户敲按键1，键盘硬件给出的扫描码是0x2，内核用它做下标查表，得到字符1的ASCII码 0x30。

按键，扫描码 和 字符码 (按下Shift键，makecode)

```
char Keyboard::Shift_Keymap[] = {
```

```
    0,0x1b,!,',@,'#','$','%','^', /* 0x00-0x07
    '&','*','(',')','_','+',0x8,0x9, /* 0x08-0x0f
    'q','w','e','r','t','y','u','i', /* 0x10-0x17
    'o','p','{','}','\n',0,'a','s', /* 0x18-0x1f
    'd','f','g','h','j','k','l',':', /* 0x20-0x27
    '\",'~',0,'|','z','x','c','v', /* 0x28-0x2f
    'b','n','m','<','>','?',0,'*', /* 0x30-0x37
    0,',',0,0,0,0,0,0, /* 0x38-0x3f
    0,0,0,0,0,0,0,0, /* 0x40-0x47
    0,0,'-',0,0,0,'+',0, /* 0x48-x04f
    0,0,0,0x7f/*del*/,0,0,0,0 /* 0x50-0x57
```

```
0, <esc> ,!,@, #,$,%,^ */
&,*,(,),_,+,<backspace>,<tab> */
q,w,e,r,t,y,u,i, */
o,p,[,], <enter>,<ctrl>,a,s, */
d,f,g,h,j,k,l,:, */
',`,<lshift>,\,z,d,c,v, */
b,n,m,,,,/,<rshif>,<printscr>, */
<alt>,<space>,<caps>,<f1>,<f2>,<f3>,<f4>,<f5>, */
<f0>,<>,<>,<>,<>,<>,<numlock>,<scrlock>,7, */
8,9,-,4,5,6,+,1, */
2,3,0,.,<>,<>,<>,<>, */
```

}; // 例：按键 Shift + 1，用户输入的是 !，字符码是 0x21。对一下你的键盘，看按键1。

这里发生的事情是：硬件先后送出2个扫描码，第一个（precode）是0x2a（按键Shift），第二个（当前scancode）是0x02（按键1）。第二次执行的键盘中断处理程序，识别出 输入字符！。

键盘码集

- 基本扫描码（单码）：key_map、Shift_Keymap数组中出现的按键，键盘硬件提供一个扫描码，是其下标。
- 扩展扫描码（双码或三码）：用来表述未出现在这两个数组的按键。
 - 表示双码，键盘硬件会依次在数据寄存器中放2个码。先是前缀e0，读取后，再放一个码。通常，第二个码会借用其它按键的扫描码。
 - Unix V6++键盘有2个双码表示的按键，右ALT(0xe0, 0x38) 和 右Ctrl (0xe0, 0x1d)，其中，0x38和 0x1d 有借用左alt、左ctrl的扫描码。以右ALT为例，(0xe0, 0x38) 是Make Code，Break Code是(0xe0, 0xb8)。
 - 表示三码，键盘硬件会依次在数据寄存器中放3个码。譬如，Pause键：先是前缀0xe1，读取后，再放一个0x1d，读取后，再放一个0x45。



键盘中断入口程序 (0x21#中断)

```
void KeyboardInterrupt::KeyboardInterruptEntrance()
{
    SaveContext();           /* 保存中断现场 */
    SwitchToKernel();        /* 进入核心态 */

    CallHandler(Keyboard, KeyboardHandler); /* 调用键盘中断处理程序 */

    IOPort::OutByte(Chip8259A::MASTER_IO_PORT_1, Chip8259A::EOI); /* 向8259A发EOI命令 */

    例行调度

    RestoreContext();        /* 恢复现场 */
    Leave();                 /* 销毁栈帧 */
    InterruptReturn();        /* 中断返回 */
}
```



键盘中断处理程序 KeyboardHandler

- (1) 识别按键
- (2) 调用函数HandleScanCode()
对按键进行处理

```
void Keyboard::KeyboardHandler( struct pt_regs* reg, struct pt_context* context )
{
    unsigned char status = IOPort::InByte(Keyboard::STATUS_PORT); // 读键盘状态端口
    int limit = 10; // 最多3码, 所以3足够了
    static int pre_state = 0; // 按键识别开始

    while ( (status & Keyboard::DATA_BUFFER_BUSY) && limit-- ) // 数据端口有新的扫描码
    {
        unsigned char scancode = IOPort::InByte(Keyboard::DATA_PORT); // 读数据端口
        if ( 0 == pre_state ) // 第一个码
        {
            if ( 0xE0 == scancode || 0xE1 == scancode ) // 二码, 0xE0。三码, 0xE1
                pre_state = scancode;
            else // 单码
            {
                pre_state = 0; // 归位
                Keyboard::HandleScanCode(scancode, 0); // 处理单码按键<0, scancode>
            }
        }
        else if ( 0xE0 == pre_state ) // 双码, 第二个扫描码
        {
            pre_state = 0; // 归位
            Keyboard::HandleScanCode(scancode, 0xE0); // 处理双码按键<0xE0, scancode>
        }
        else if ( 0xE1 == pre_state && ( 0x1D == scancode || 0x9D == scancode ) ) // 三码, 第二个码
        {
            pre_state = 0x100;
        }
        else if ( pre_state == 0x100 && 0x45 == scancode ) // 三码, 第三个码
        {
            pre_state = 0; // 归位
            Keyboard::HandleScanCode(scancode, 0xE1); // 处理三码按键<0xE1, 0x45>
        }
        else
            pre_state = 0;
        status = IOPort::InByte(Keyboard::STATUS_PORT); // 读状态端口
    }
}
```



1、识别出 Pause 键

```
void Keyboard::HandleScanCode(unsigned char scanCode, int expand)
{
    int isOK = 0;
    char ch = 0;

    if ( 0xE1 == expand )
    {
        ch = Keyboard::ScanCodeTranslate(scanCode, expand); // Pause
    }
}
```

1

2、识别出 Shift/ALT/Ctrl 用全局变量Mode记录下来 这些控制键处于按下状态

```
switch ( scanCode )
{
case SCAN_ALT:
    if ( 0xE0 == expand ) /* 使用扩展键，表示是右边的 alt按下 */
        Mode |= M_RALT;
    else
        Mode |= M_LALT;
    break;

case SCAN_CTRL:
    if ( 0xE0 == expand ) /* 使用扩展键，表示是右边的 ctrl按下 */
        Mode |= M_RCTRL;
    else
        Mode |= M_LCTRL;
    break;

/* 处理按键被松开的情况，清空Mode中按键对应的标志位 */
case SCAN_LSHIFT: // 左Shift按下
    Mode |= M_LSHIFT;
    break;

case SCAN_RSHIFT: // 右Shift按下
    Mode |= M_RSHIFT;
    break;

case SCAN_CTRL + 0x80:
    if ( 0xE0 == expand )
        Mode &= ~M_RCTRL;
    else
        Mode &= ~M_LCTRL;
    break;

case SCAN_LSHIFT + 0x80:
    Mode &= ~M_LSHIFT;
    break;

case SCAN_RSHIFT + 0x80:
    Mode &= ~M_RSHIFT;
    break;
}
```

2

3

3、识别出 Shift/ALT/Ctrl 用全局变量Mode记录下来 这些控制键松开了

```
/* 每次按下就反转改变状态位 */

/*
 * M_DOWN_NUMLOCK宏表示NUMLOCK被按下，在没有按下的
 * 状态下按下NumLock键需要反转状态，并置numlock键没有松开
 */
case SCAN_NUMLOCK:
    isOK = Mode & M_DOWN_NUMLOCK;
    if ( !isOK )
    {
        Mode ^= M_NUMLOCK;
        Mode |= M_DOWN_NUMLOCK;
    }
    break;

case SCAN_CAPSLOCK:
    isOK = Mode & M_DOWN_CAPSLOCK;
    if ( !isOK )
    {
        Mode ^= M_CAPSLOCK;
        Mode |= M_DOWN_CAPSLOCK;
    }
    break;

case SCAN_SCRLOCK:
    isOK = Mode & M_DOWN_SCRLOCK;
    if ( !isOK )
    {
        Mode ^= M_SCRLOCK;
        Mode |= M_DOWN_SCRLOCK;
    }
    break;

/* 释放NumLock, CapsLock, ScrollLock键 */
case SCAN_NUMLOCK + 0x80:
    Mode &= ~M_DOWN_NUMLOCK;
    break;

case SCAN_CAPSLOCK + 0x80:
    Mode &= ~M_DOWN_CAPSLOCK;
    break;

case SCAN_SCRLOCK + 0x80:
    Mode &= ~M_DOWN_SCRLOCK;
    break;
}
```

4

4、开启 CapsLock 大写输入模式，用NumLock 开启小键盘，输入数字

~~~

关闭这些键盘工作模式

## HandleScanCode( )处理扫描码

- (1) 识别Shift、Ctrl、Alt、CapsLock、NumberLock等控制键，修改键盘工作模式。
- (2) 识别MakeCode、过滤BreakCode
- (3) 调用函数ScanCodeTranslate( )，查转换表。
  - ctrl + c，向所有进程（除 0#、1#进程之外）发SIGINT信号，杀死这些进程。
  - 其余字符，获得其ASCII码，送终端字符缓存（马上会说）。
- (4) 字符码送缓存队列

```
default:
    ch = Keyboard::ScanCodeTranslate(scanCode, expand);
    break;
}
```

5

```
if ( 0 != ch )
{
    Tty* pTty = Kernel::Instance().GetDeviceManager().GetCharDevice(DeviceManager::TTYDEV).m_TTy;
    if ( NULL != pTty )
    {
        pTty->TTYInput(ch);
    }
}
```

6

## 5、翻译扫描码

## 6、字符码送缓存队列



# ScanCodeTranslate( )翻译扫描码

```
char Keyboard::
ScanCodeTranslate(unsigned char scanCode, int expand)
```

```
{
    char ch = 0;
    bool bReverse = false;

    if ( 0xE1 == expand )      /* Pause Key */
    {
        ch = 0x05;          /* Pause ASCII */
    }
}
```

1

```
else if ( scanCode < 0x58 ) // 按NmLk键，用小键盘。
{
    bReverse = ( (Mode & M_NUMLOCK) ? 1 : 0 ) ^ ( (Mode & M_LSHIFT) || (Mode & M_RSHIFT) );

    if ( 0xE0 == expand )
    {
        ch = Shift_Keymap[scanCode];
    }
    else if ( bReverse )
    {
        ch = Keymap[scanCode];
    }
    else
    {
        ch = Shift_Keymap[scanCode];
    }
}

return ch; // 返回按键对应的ASCII码。0表示没有需要送入tty缓存的字符。
}
```

3

数字小键盘

```
else if ( scanCode < 0x45 ) /* 非小键盘和控制键 */
{
    /* 根据扫描码映射表找到对应按键的ASCII码 */
    if ( (Mode & M_LSHIFT) || (Mode & M_RSHIFT) )
    {
        ch = Shift_Keymap[scanCode];
    }
    else
    {
        ch = Keymap[scanCode];
    }

    if ( ch >= 'a' && ch <= 'z' )
    {
        /* Capslock 或 Shift 小写，要转换成大写字符 */
        bReverse = ( (Mode & M_CAPSLOCK) ? 1 : 0 ) ^ ( (Mode & M_LSHIFT) || (Mode & M_RSHIFT) );

        if ( (Mode & M_LCTRL) || (Mode & M_RCTRL) ) /* 按下ctrl进行转义 */
        {
            if ('c' == ch) /* ctrl+c --> SIGINT信号，送给除 sched、shell 之外的所有进程 */
            {
                ch = 0;

                /* FLush终端 */
                TTY* pTTY = Kernel::Instance().GetDeviceManager().
                    GetCharDevice(DeviceManager::TTYDEV).m_TTy;
                if ( NULL != pTTY )
                {
                    pTTY->FlushTTY();
                }

                ProcessManager& procMgr = Kernel::Instance().GetProcessManager();
                for ( int killed = 0; killed < ProcessManager::NPROC; killed ++ )
                {
                    if ( procMgr.process[killed].p_pid > 1 )
                        procMgr.process[killed].PSignal(User::SIGINT);
                }
            }
            else
            {
                ch -= 'a';
                ch++; /* 转义从0x1 开始，暂时没有用，不管它 */
            }
        }
        else if ( bReverse )
        {
            ch += 'A' - 'a';
        }
    }
}
}
```

查转换表，变大写.....  
ctrl+c 实施进程控制

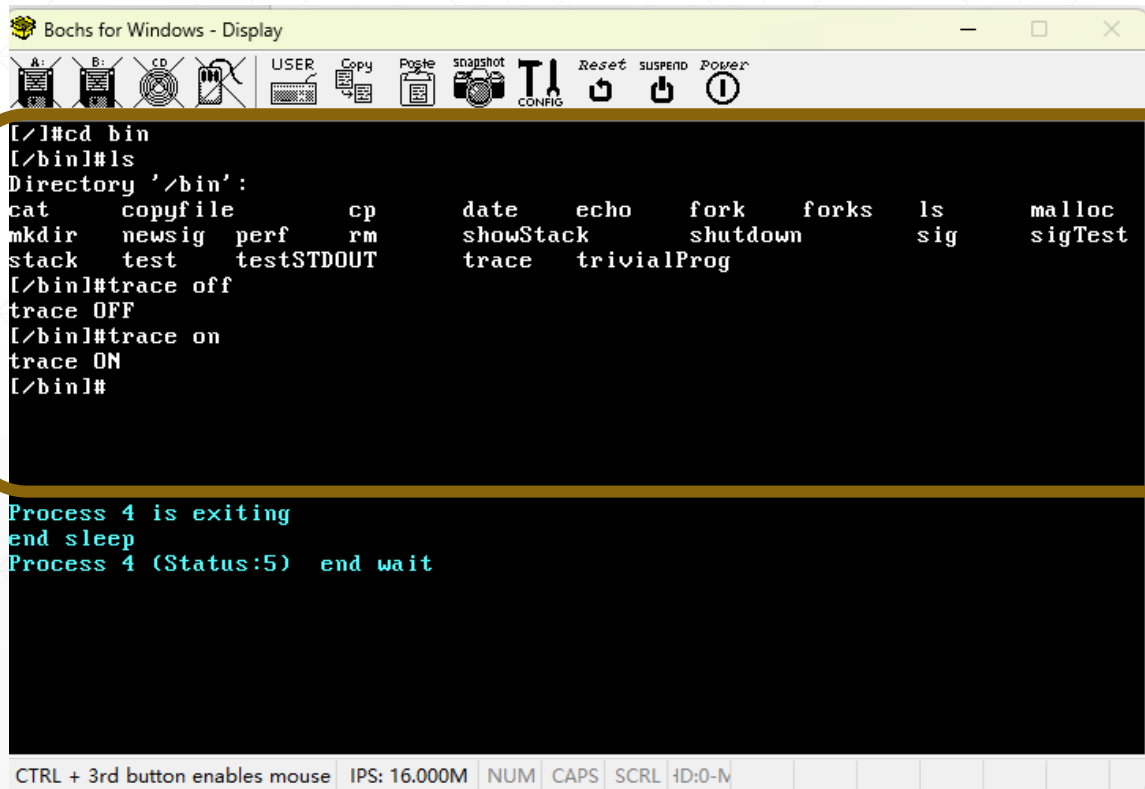
2



## Part 3、TTY缓存 和 应用程序的标准输入输出

# 人机接口 和 应用程序的标准输入输出

Unix V6++的人机接口，一个字符终端 (terminal)



```

Bochs for Windows - Display
[ / ] # cd bin
[ / bin ] # ls
Directory '/bin':
cat      copyfile      cp      date      echo      fork      forks      ls      malloc
mkdir    newsig    perf     rm      showStack shutdown sig      sigTest
stack    test      testSTDOUT
[ / bin ] # trace off
trace OFF
[ / bin ] # trace on
trace ON
[ / bin ] #

Process 4 is exiting
end sleep
Process 4 (Status:5) end wait

CTRL + 3rd button enables mouse  IPS: 16.000M  NUM  CAPS  SCRL 4D:0-M

```



包括一个键盘和一个显示器（严格说来，只包括屏幕上的应用程序显示区），没有鼠标

# tty结构

- tty结构是Unix内核的终端控制块。一个tty结构管理一个终端，为一个用户服务。Unix V6++是PC（Personal Computer），只有一个用户，所以一个终端、一个tty。
- 每个tty结构中，最重要的是3个缓存队列：
  - 原始队列，放键盘输入的字符
  - 标准队列，放用户确认的输入字符
  - 输出队列，放应用程序输出的字符串

设置缓存以后，集齐一行才会唤醒睡眠等待键盘输入的应用程序。这可以减少字符终端IO对系统的干扰、提高计算/IO并发度；还能降低进程入睡、唤醒和进程切换次数，改善系统运行效率。



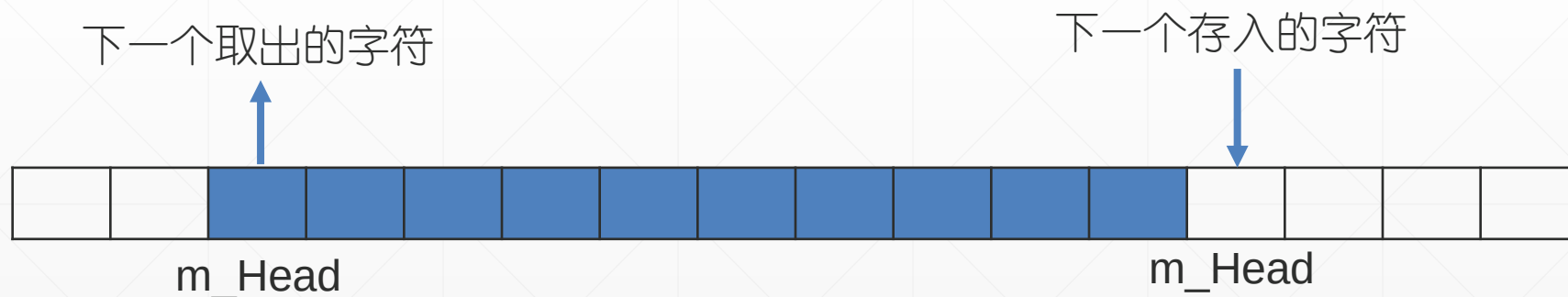
```
class TTy
{
public:
    static const unsigned int CANBSIZ = 256;

public:
    short dev;                /* 设备号 (major, minor) == (0, 0) */
    TTy_Queue t_rawq;         /* 原始队列*/
    TTy_Queue t_canq;         /* 标准队列*/
    TTy_Queue t_outq;         /* 输出队列 */
    char Canonb[CANBSIZ];     /* 工作缓存, 用来处理t_rawq中的字符 */

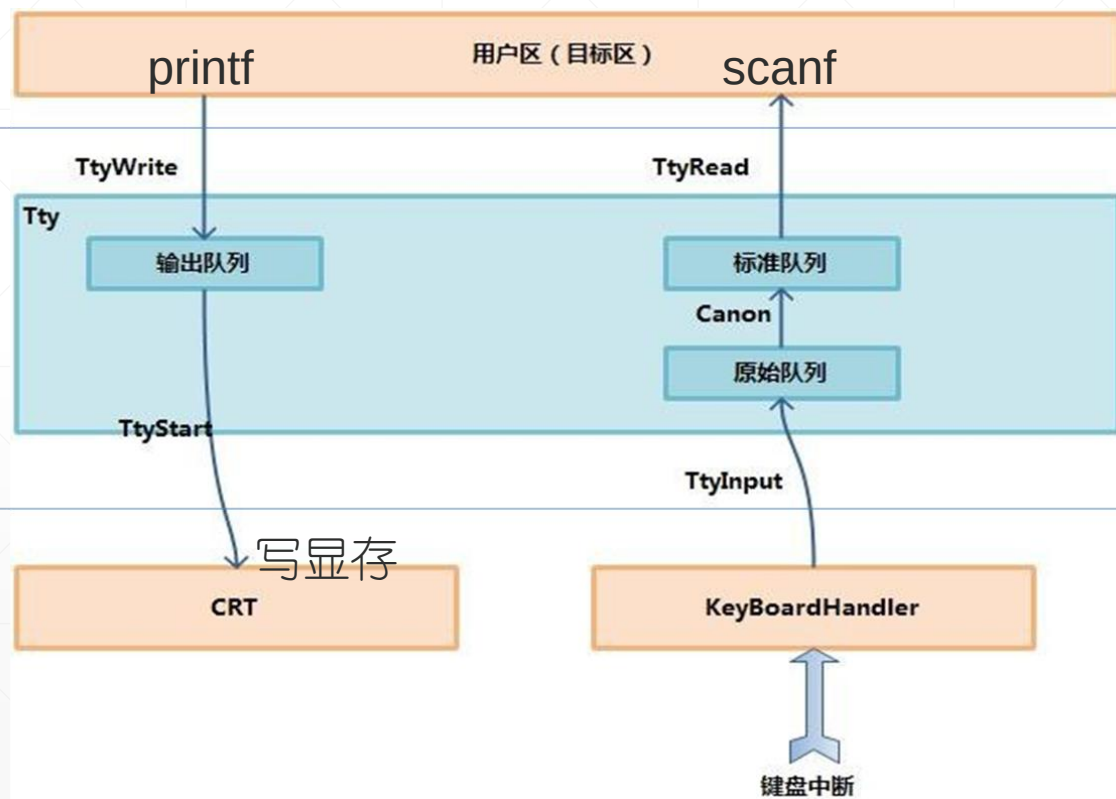
    int t_flags;              /* 字符设备工作标志字 */
    int t_delct; /* 原始队列中的定界符数 (定界符标记一行的结束, Unix V6++是0x7, 追加在回车符'\n'之后) */
    char t_erase;             /* 擦除键字符 (BackSpace) */
    char t_kill;              /* 删除整行字符 */
    int t_state;              /* 设备状态字 */

};
```

```
class TTy_Queue // 字符缓存队列。FIFO循环队列。
{
public:
    static const unsigned int TTY_BUF_SIZE = 512;
public:
    unsigned int m_Head;    /* 队首 */
    unsigned int m_Tail;    /* 队尾 */
    char m_CharBuf[TTY_BUF_SIZE]; /* 字符缓存数组，当循环队列用 */
};
```



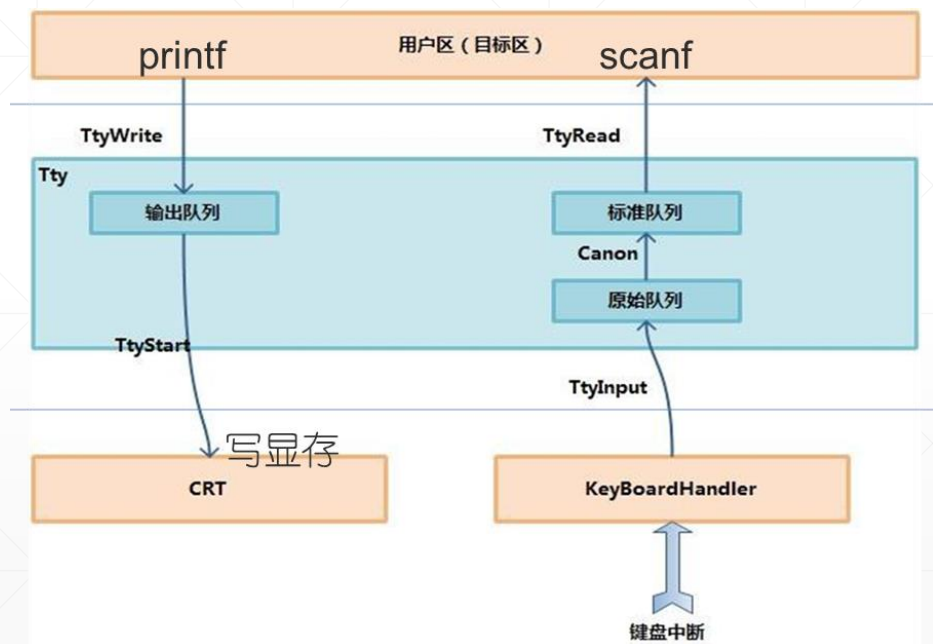
# 缓存队列的功能



原始队列、标准队列 和 输出队列是内核缓存，介于应用程序和硬件驱动之间

原始队列 **t\_rawq**，存放从键盘获得的原始输入字符  
标准队列 **t\_canq**，原始队列集齐一行后，系统会运用标准的行处理规则处理其中的控制字符，比如退格 (backspace) 擦除光标之前的字符。  
输出队列 **t\_outq**，存放向CRT输出的字符。

# 缓存队列的使用规则



- 每敲一下键盘，键盘中断处理程序KeyBoardHandler就会执行一次。它读输入字符，送t\_rawq 尾部（马上会说回显）。
- 应用程序执行scanf()或getc()，进程读标准队列。若标准队列为空，进程入睡。会睡到什么时候呢？用户输入回车符。
- 回显。当用户敲键盘的时候，系统需要回显用户输入。也就是说，中断处理程序获得用户输入的字符之后，除了送它进原始队列之外，还要把它塞到输出队列，并立即写入显存。
- 应用程序执行printf(), putc(), 进程将一批需要显示的字符写入输出队列。如果这批字符中有回车符，无论有多少，最后一个回车符之前的所有字符会立即显示在屏幕上。之后的所有字符不会出现在屏幕上，它们留在输出队列中，等待printf送来下一个回车符（这是正确的逻辑，Unix V6++打了折扣，最后一行没有回车也会显示在屏幕上）。



# 缓存队列基本操作

```
char TTy_Queue::GetChar()
{ 1、返回队尾字符, 指针m_Tail向后移 1 位
  char ch = TTy::GET_ERROR;
  {
    if ( this->m_Head == this->m_Tail )
      return ch; // 队列为空时, 返回 -1
  }
  ch = this->m_CharBuf[m_Tail];
  this->m_Tail = ( this->m_Tail + 1 ) % TTY_BUF_SIZE;
  return ch;
}
```

```
void TTy_Queue::PutChar(char ch)
{ 2、ch 存入队首, m_Head向后移 1 位
  this->m_CharBuf[m_Head] = ch;
  this->m_Head = ( this->m_Head + 1 ) % TTY_BUF_SIZE;
}
```

```
char* TTy_Queue::CurrentChar()
{ 3、返回队尾指针
  return &this->m_CharBuf[m_Tail];
}
```

```
int TTy_Queue::CharNum()
{ 4、返回缓存中字符数量
  int ans = (this->m_Head - this->m_Tail)
    & (TTy_Queue::TTY_BUF_SIZE - 1);
  return ans;
}
```

# TTYRead() 终端读操作: 读键盘, 获得应用程序 (scanf函数) 需要的所有字符

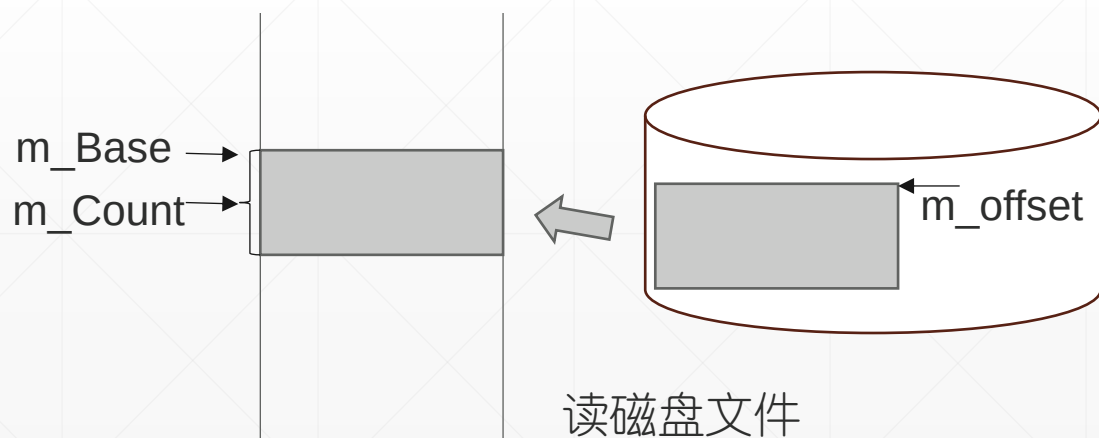
```
void TTy::TTYRead()  
{ // 读标准队列 t_canq, 直至满足应用程序数量要求 (下  
  页PPT) 或队列取空  
  if ( (this->t_state & TTy::CARR_ON) == 0 )  
    return; // 终端 (屏幕) 没开  
  
  if ( this->t_canq.CharNum() || this->Canon() )  
    while ( this->t_canq.CharNum() &&  
            (this->PassC(this->t_canq.GetChar()) >= 0) );  
}
```

如果开始时标准队列空, 则调用Canon()程序从原始队列取一行数据 (准确说是第一个定界符0x7之前的全部字符)。如果原始队列没有定界符 (本质上是回车符), 意味着用户尚未确认输入, 进程睡眠等待。

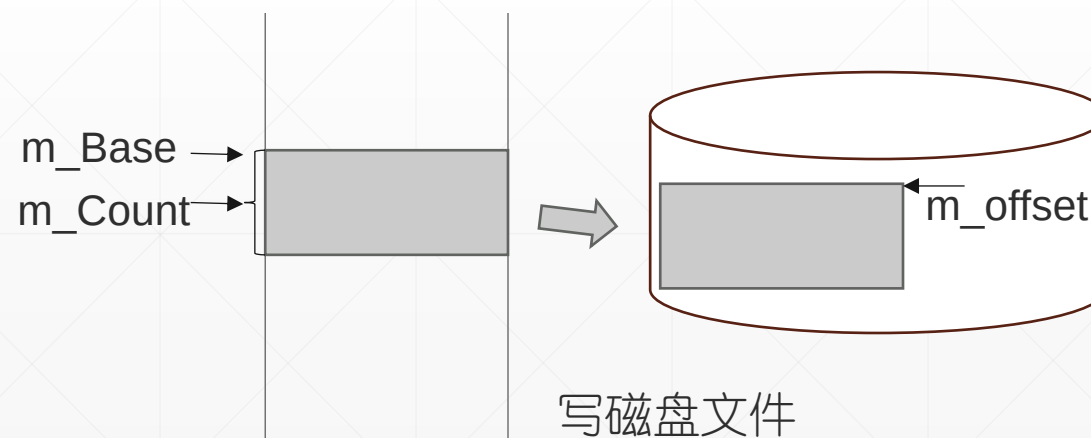
```
int TTy::PassC(char ch)  
{ // 将字符ch送入用户空间, m_Base指向的内存单元  
  User& u = Kernel::Instance().GetUser();  
  
  if ( u.u_IOParam.m_Count > 0 )  
  {  
    *(u.u_IOParam.m_Base++) = ch;  
    u.u_IOParam.m_Count--;  
    return 0; // .....更新IO参数  
  }  
  return -1;  
}
```

## 旁白：Unix内核的IO参数，读写磁盘文件

- 进程的 user 结构中，有一个 u IOParam 字段。读写磁盘文件时，read、write 系统调用会首先设置磁盘字符块的IO参数，包括用户空间为其分配的内存块的首地址和长度 以及 磁盘数据块在磁盘文件中的偏移量。具体而言：
  - m\_Base, 内存块首地址
  - m\_Count, 内存块的尺寸
  - m\_Offset, 数据块在文件中的偏移量



读磁盘文件



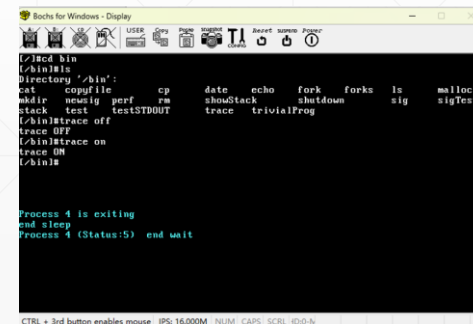
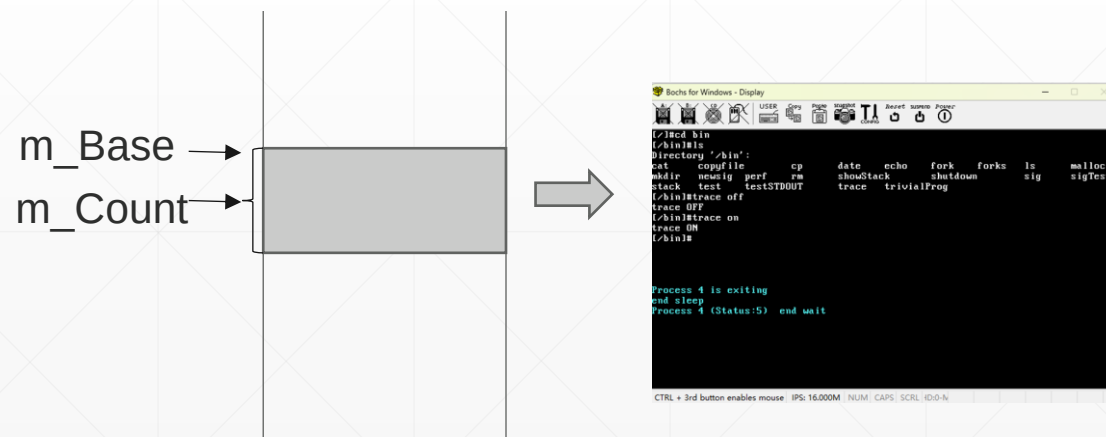
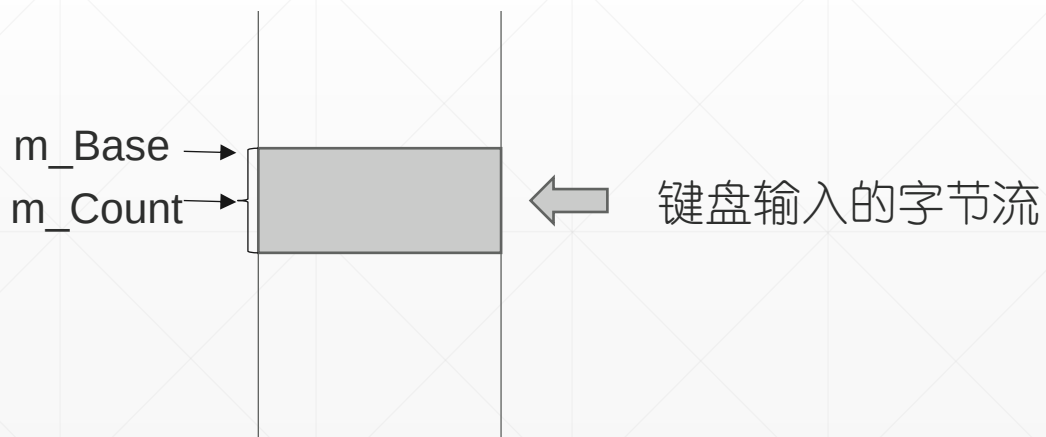
写磁盘文件

## 旁白：Unix内核的IO参数，读键盘、写屏幕

- 读写标准输入输出文件时，read、write 系统调用也要设置IO参数。因为不是读写磁盘文件，所以IO参数中没有m\_Offset。

- m\_Base, 内存块首地址
- m\_Count, 内存块的尺寸

read系统调用 FIFO 地读入用户键入的字符，write系统调用将内存块中的所有字节显示在屏幕上。



A screenshot of a terminal window titled 'Tochu for Windows - Display'. It shows a list of system calls and their corresponding functions: cat, copyfile, cp, date, echo, fork, forks, is, malloc, mkdir, newsig, perf, rm, showStack, shutdown, sig, sigTest, stack, test, testSTROUT, trace, trivialProg. Below this, it shows 'trivialProg' being executed. At the bottom, it says 'Process 4 is exiting' and 'end sleep'.

```
void TTy::TTWrite()
{
```

```
    char ch;
```

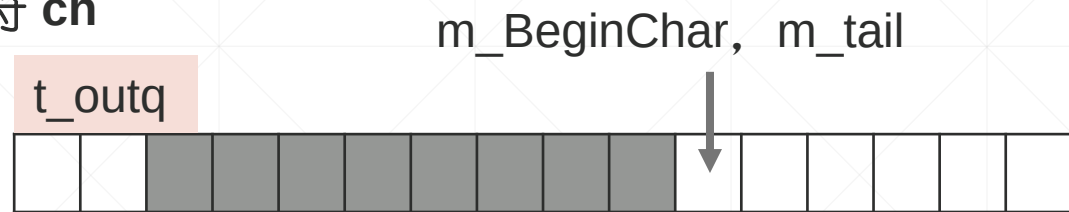
```
    if ( (this->t_state & TTy::CARR_ON) == 0 )
        return; // tty不工作（屏幕没开）
```

```
    while ( (ch = CPass()) > 0 ) // 循环，每次从用户空间取一个字符 ch
```

```
    {
        if ( this->t_outq.CharNum() > TTy::TTHIWAT) // t_outq满啦
        {
            this->TTStart(); // 将队列中的所有字符写进显存
            CRT::m_BeginChar = this->t_outq.CurrentChar();
        }
        this->TTyOutput(ch); // 将ch写入t_outq
    }
```

```
    this->TTStart(); // 将剩余字符全部显示在屏幕上
    CRT::m_BeginChar = this->t_outq.CurrentChar();
}
```

## TTWrite()终端写操作： 写屏幕，显示应用程序 (printf) 输出的字符串



图：阴影部分是队列中已有的字符。TTStart()将其全部送入显存(显示)后， m\_BeginChar 在这个位置。

显存中，这个位置之前的字符已被确认，除非屏幕滚动和清屏，已确认字符不可擦除和更改。

所以， m\_BeginChar的作用在于（1）write系统调用执行阶段，定位缓存中TTStart()下次执行时输出的第一个字符。（2）回显键盘输入字符时，backspace可以擦除的内容到此为止。



# TTyOutput( )

```
void TTy::TTyOutput(char ch)
{
    /* 将字符 ch 放入输出缓存队列中 */
    if (ch)
        this->t_outq.PutChar(ch);
}
```



# TTStart(), 显示输出缓存中的内容

crt.h

```
#include "TTy.h"
class CRT
{
```

```
    static const unsigned short VIDEO_ADDR_PORT = 0x3d4;
    static const unsigned short VIDEO_DATA_PORT = 0x3d5;
    /* 屏幕大小为80 * 25 */
    static const unsigned int COLUMNS = 80; // 80列
    static unsigned int ROWS;
    static const unsigned short COLOR = 0x0F00; // 黑底白字
```

```
    static unsigned short* m_VideoMemory; // 显存首地址
    static unsigned int m_CursorX; // 光标的坐标
    static unsigned int m_CursorY;
```

```
    static char* m_Position; // tty输出缓存中要显示的下一个字符，每输出一个字符m_Position++，如果是backspace，m_Position--。
    static char* m_BeginChar; // tty输出缓存中，BackSpace不能擦掉这根指针之前的字符（这根指针之前所有字符已确认。已确认
    字符包括应用程序输出的字符，用户用回车确认的命令行 或 应用程序输入）。
    .....
};
```

关键字段

crt.cpp

```
#include "CRT.h"
#include "IOPort.h"
```

```
unsigned int CRT::ROWS = 15;
unsigned short* CRT::m_VideoMemory =
    (unsigned short*)(0xB8000 + 0xC0000000);
unsigned int CRT::m_CursorX = 0; // 光标的初始位置
unsigned int CRT::m_CursorY = 0;
char* CRT::m_Position = 0;
char* CRT::m_BeginChar = 0;
.....
```



```
void TTy::TTStart()
{
    CRT::CRTStart(this);
}
```

```
void CRT::CRTStart(TTy* pTTY)
{
    char ch;
    if ( 0 == CRT::m_BeginChar)
    {
        m_BeginChar = pTTY->t_outq.CurrentChar();
    }
    if ( 0 == m_Position )
    {
        m_Position = m_BeginChar;
    }
}
```



```
while ( (ch = pTTY->t_outq.GetChar())!= TTy::GET_ERROR ) // t_outq中取一个字符
{
    switch (ch)
    {
        case '\n': // 回车符表示确认
            NextLine();
            CRT::m_BeginChar = pTTY->t_outq.CurrentChar(); // 同步调整m_BeginChar 和 m_Position
            m_Position = CRT::m_BeginChar;
            break;

        .....
        case '\b': // Backspace
            if ( m_Position != CRT::m_BeginChar )
            {
                BackSpace(); // 擦显示屏上的字符
                m_Position--; // 退一格
            }
            break;

        case '\t':
            Tab();
            m_Position++; // 进一格
            break;

        default:
            WriteChar(ch); // 屏幕显示普通字符 */
            m_Position++; // 进一格
            break;
    }
}
}
```



## TTYInput(ch), 磁盘中断处理程序将输入字符 ch 放入缓存

```
void TTy::TTYInput(char ch)
{
    this->t_rawq.PutChar(ch);           // 将输入字符 ch 放入原始队列

    if ( ch == '\n' || ch == TTy::CEOT ) // 回车符之后插入定界符0x7, 标记一行结束; 原始队列的字符行, 数量加1
    {
        Kernel::Instance().GetProcessManager().WakeUpAll((unsigned long)&this->t_rawq);
        this->t_rawq.PutChar(0x7);
        this->t_delct++;
    }

    if ( this->t_flags & TTy::ECHO ) // 需要回显用户输入
    {
        this->TTYOutput(ch); // 送输出队列
        this->TTStart(); // 立即显示
    }
}
```

唤醒等待键盘输入的进程  
呼应 PPT31 的入睡操作



# FlushTTy(), 刷新所有缓存

```
void TTy::FlushTTy()
{
    while ( this->t_canq.GetChar() >= 0 );
    while ( this->t_outq.GetChar() >= 0 );
    Kernel::Instance().GetProcessManager().WakeUpAll((unsigned long)&this->t_canq);
    Kernel::Instance().GetProcessManager().WakeUpAll((unsigned long)&this->t_outq);

    X86Assembly::CLI();
    while ( this->t_rawq.GetChar() >= 0 );
    this->t_delct = 0;
    X86Assembly::STI();
}
```



# Canon(), call by read系统调用: 从原始队列取一行, 运用行处理规则处理这一行字符, 完成后, 存入标准队列 t\_canq, 供应应用程序读取

```
int TTy::Canon()
{
    char* pChar;
    char ch;
    User& u = Kernel::Instance().GetUser();

    X86Assembly::CLI();
    while ( this->t_delct == 0 ) // 原始队列无回车符, 没有可以送给应用程序的数据, 睡眠等待用户输入回车符确认输入
    {
        if ( (this->t_state & TTy::CARR_ON) == 0 )
            return 0; // 设备没打开, 返回
        u.u_procp->Sleep((unsigned long)&this->t_rawq, ProcessManager::TTIPRI);
    }
    X86Assembly::STI();
}
```

PPT29 的Wakeup把它唤醒



```
// 处理原始队列中的字符，送入canon队列
pChar = &Canonb[0];

while ( (ch = this->t_rawq.GetChar()) >= 0 ) // 从原始队列取一行字符，存入Canonb队列。
{
    if ( 0x7 == ch ) /* 是定界符*/
    {
        this->t_delct--; // 原始队列行数--
        break;
    }

    if ( ch == this->t_erase ) /* 是backspace */
    {
        if ( pChar > &Canonb[0] )
            pChar--; // 擦掉Canonb中的前一个字符。Yeah !!! 删除屏幕上的字符，这里有一半
        continue;
    }

    if ( ch == TTy::CEOT ) /* CEOT == 0x4 (试试 ctrl + d) */
        continue; /* 是文件结束符。没数据啦，返回 */

    *pChar++ = ch; /* 是普通字符。存入Canonb队列 */

    if ( pChar >= Canonb + TTy::CANBSIZ )
        break; /* Canonb队列满返回。本行剩余字符，下次Canon函数执行时再取 */
}

char* pEnd = pChar;
pChar = &Canonb[0];

while ( pChar < pEnd )
    this->t_canq.PutChar(*pChar++); /* 将Canonb队列中处理过的字符送标准队列 */

return 1;
}
```

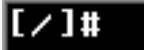


## Part 4、读写标准输入输出文件的read, write系统调用

# 应用程序的标准输入、输出文件 STDIN 和 STDOUT

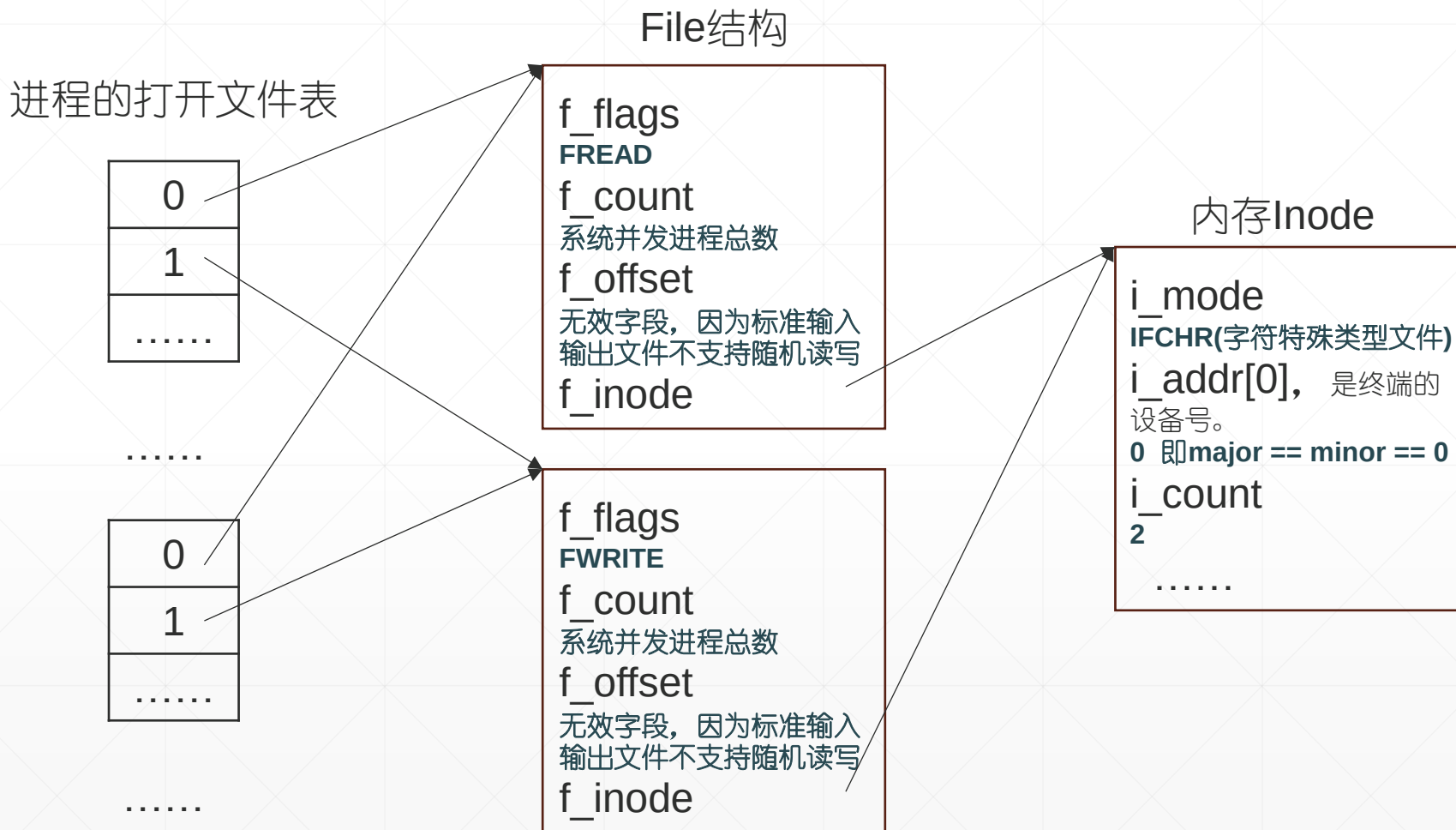
- 每个应用程序，有1个标准输入文件（0#文件） 和 一个标准输出文件（1#文件），分别是 键盘 和 屏幕。
- 与磁盘文件不同，使用标准输入输出文件之前，不需要执行open操作。Unix V6++系统初始化时，main0打开这两个文件，让它们成为0#进程打开文件表中的0# 和 1#文件。

```
int fd_tty;  
fd_tty = lib_open("/dev/tty1", File::FREAD);  
fd_tty = lib_open("/dev/tty1", File::FWRITE);    // 没错，/dev/tty1是Unix V6++终端的文件名，  
                                                  回顾上学期 格式化文件系统的作业。
```

- 0#进程创建1#进程，1#进程继承标准输入输出文件，可以读键盘，可以写屏幕。故exec(shell)之后，该shell进程执行 read(0, ..... ) 读用户键盘输入的命令行、write(1, ..... ) 向屏幕输出命令行提示符：

- 其余进程是shell进程的子进程。它们用同样的方式继承、访问标准输入输出文件。

# 标准输入输出文件的打开文件结构





# 应用程序执行Write系统调用，写显示屏与用户交互

**流程** `printf` → `write( 1, ..... )`  
将用户空间的字符写入tty的输出缓存队列 → 集齐一行写入显存

CRT定时读取显存中所有字符，刷新屏幕  
`printf`输出的字符串出现在屏幕上

```
void printf(char* fmt, ... )
{
    char buffer[1024];
    va pva = getvahead(fmt);
    int count = _sprintf(buffer, fmt, pva);
    write(STDOUT, buffer, count);
    写1#, 标准输出文件
}
```

**1 Write系统调用, Rdwr( )** ..... **2**  
`pFile = u.u_ofiles.GetF(u.u_arg[0]);` // 取STDOUT对应的File结构  
`pFile->f_inode->Writel();` // 通过这个File结构, 找到终端文件tty的内存Inode

**Writel( ) // Unix V6++**, 所有类型的文件共用同一个Write函数 ..... **3**  
`u.u_IOParam.m_Base = (unsigned char *)u.u_arg[1];` // 首地址  
`u.u_IOParam.m_Count = u.u_arg[2];` // 长度  
`u.u_segflg = 0;` // 1、设置IO参数: 需要送CRT显示的数据块, 首地址和长度  
`if( (this->i_mode & Inode::IFMT) == Inode::IFCHR )`  
{ // 2、用设备号中的major定位、执行字符设备开关表中的write函数  
`short major = Utility::GetMajor(this->i_addr[0]);`  
`devMgr.GetCharDevice(major).Write(this->i_addr[0]);`  
`return;`  
}

辅助函数GetCharDevice(short major)  
return \*(this->cdevsw[major]);

**4**  
`void ConsoleDevice::Write(short dev)`  
`short minor = Utility::GetMinor(dev);`  
`if ( 0 == minor )`  
`this->m_TTy->TTWrite();`

// Write函数用设备号中的minor, 定位tty, 找到输出缓存。执行TTWrite( )在屏幕上写字符串

其它类型文件的写操作分支, 期末考的写磁盘文件在这里

# 应用程序执行Read系统调用，读键盘输入

**流程** scanf → read( 0, ..... ) → 进程读tty的标准队列 如果标准队列和原始队列没有回车，Sleep入睡

用户敲键盘，键盘中断处理程序送ASCII字符进原始队列，  
如果这个字符是回车，唤醒睡眠进程（上面这个）取原始队列中的数据

**void printf(char\* fmt, ... )**

```
{
    char buffer[1024];
    va pva = getvahead(fmt);
    int count = _sprintf(buffer, fmt, pva);
    write(STDOUT, buffer, count);
}
```

写1#，标准输出文件

1

**Write系统调用, Rdwr() .....**

pFile = u.u\_ofiles.GetF(u.u\_arg[0]); // 取STDOUT对应的File结构  
pFile->f\_inode->**Writel();** // 通过这个File结构，找到终端文件tty的内存Inode

2

**Writel() // Unix V6++，所有类型的文件共用同一个Write函数 .....** 3

u.u\_IOParm.m\_Base = (unsigned char \*)u.u\_arg[1]; // 首地址  
u.u\_IOParm.m\_Count = u.u\_arg[2]; // 长度  
u.u\_segflg = 0; // 1、设置IO参数：需要送CRT显示的数据块，首地址和长度  
if( (this->i\_mode & Inode::IFMT) == Inode::IFCHR )  
{  
 short major = Utility::GetMajor(this->i\_addr[0]);  
 devMgr.GetCharDevice(major).**Write(this->i\_addr[0]);**  
 return; // 2、用设备号中的major定位、执行字符设备开关表中的write函数  
}

其它类型文件的写操作分支，期末考的写磁盘文件在这里

辅助函数GetCharDevice(short major)  
return \*(this->cdevsw[major]);

**void ConsoleDevice::Write(short dev)** 4

short minor = Utility::GetMinor(dev);  
if ( 0 == minor )  
 this->m\_TTy->TTWrite();

// Write函数用设备号中的minor，定位tty，找到输出缓存。执行TTWrite()在屏幕上写字符串



读键盘输入也有这一套  
同学们自己理出来

加油，加油